

PLATYPULSE

PROJECT REPORT

Harsh Thorat

Lucky Belel

August 2026

Acknowledgement

We are extremely grateful to our mentors – Lakshay Lalwani, Pushkar Dube and Sarvaath Narang for their guidance and support throughout the course of this project.

We also express gratitude towards SRA-VJTI for their support as well as organization of ‘Eklavya - 2026’ and providing us with the opportunity to work on this project.

Harsh Thorat
Lucky Belel

Overview

Modern technology is built upon countless forms of communication that remain invisible to the user. From remote controls and access systems to identification tags and wireless devices, information is continuously exchanged through signals that form the foundation of our connected world. Platypulse is an embedded systems project inspired by the concept of the Flipper Zero, developed to explore, understand, and interact with these communication technologies through a unified hardware platform.

The primary objective of Platypulse is to provide a compact system capable of receiving, transmitting, storing, and analyzing signals across three widely used communication domains: Radio Frequency (RF), RFID/NFC, and Infrared (IR). These technologies represent different approaches to short- and medium-range communication and are commonly found in everyday electronic and embedded systems.

Platypulse brings these technologies together into a single platform, allowing captured signals and communication data to be processed and stored for further analysis and controlled reproduction. The project combines embedded hardware, firmware, communication interfaces, and custom PCB design to create an integrated system rather than treating each technology as an independent module.

Beyond its functional objectives, Platypulse is intended as a platform for understanding how wireless communication operates at the hardware and protocol level. By observing, capturing, and interpreting signals, the project provides practical insight into concepts such as signal modulation, communication protocols, data representation, and device interoperability.

The development of Platypulse therefore represents an exploration of the boundary between the physical and digital worlds. Its purpose is not simply to reproduce existing functionality, but to provide a foundation for learning, experimentation, and responsible development of embedded wireless technologies. Through the integration of RF, RFID/NFC, and IR into one system, Platypulse demonstrates how multiple communication technologies can be brought together to create a versatile and purposeful embedded platform.

Contents

Acknowledgement	1
Overview	2
1 Introduction	6
1.1 About	6
1.2 Problem Statement	6
1.3 Solution	6
2 Custom PCB Layout	7
2.1 Introduction	7
2.2 List of Components	7
2.3 Circuit Connection	7
2.3.1 ESP32-S3-WROOM-1 Connections	7
2.3.2 CC1101 (Sub-GHz Transceiver) Connections	8
2.3.3 IR Transmitter and TSOP38238 (IR Receiver) Connections	9
2.3.4 PN532 Connections	10
2.3.5 Voltage Regulator Connections	11
2.3.6 Display TFT Connections	11
2.3.7 Joystick Connections	12
2.3.8 Modular RF Interface	12
2.4 Routing	12
3 Wireless Communication	14
3.1 Introduction	14
3.2 CC1101 — Sub-1 GHz RF Transceiver	14
3.2.1 Communication Workflow	14
3.2.2 Register Access over SPI	15
3.2.3 Modulation and Line Encoding	15
3.2.4 Error Checking	15
3.3 PN532 — NFC/RFID Module	15
3.3.1 Frame Structure	15
3.3.2 Checksums	16
3.3.3 NFC Data Handling and RFID Access-Control Observations	16
3.4 IR Signal Learning, Storage, and Emulation	17
3.4.1 Signal Learning / Receiving	17
3.4.2 Storing and Emulating	17
4 Wired Communication	18
4.1 SPI (Serial Peripheral Interface)	18
4.2 I2C (Inter Integrated Circuit)	19
5 Software Defined Radio	20
5.1 Introduction	20
5.2 Receiving The Signal	20

5.2.1	Introduction	20
5.2.2	Vector Source	20
5.2.3	Repack Bits	20
5.2.4	Repeat	20
5.2.5	Char to Float	20
5.2.6	Constant Source	20
5.2.7	Float to Complex	21
5.2.8	Multiply Const	21
5.2.9	Low Pass Filter	21
5.2.10	Osmocom Sink	21
5.3	Transmitting the Signal	21
5.3.1	Introduction	21
5.3.2	RTL-SDR Source	22
5.3.3	QT GUI Frequency Sink	22
5.3.4	QT GUI Time Sink	22
5.3.5	QT GUI Constellation Sink	22
5.3.6	Frequency Xlating FIR Filter	22
5.3.7	File Sink	22
5.3.8	Complex to Mag ²	23
5.3.9	QT GUI Time Sink (Envelope)	23
5.3.10	AGC2	23
5.3.11	File Sink (Normalized)	23
5.3.12	Threshold	23
5.3.13	QT GUI Time Sink (Digital)	23
5.3.14	Float to Char	23
5.3.15	Keep M in N	23
5.3.16	Repack Bits (Receiver)	23
5.3.17	QT GUI Time Sink (Byte Stream)	23
5.3.18	Correlate Access Code - Tag	24
6	Firmware	25
6.1	Introduction	25
6.2	IR Firmware	25
6.2.1	IR Receiving	25
6.2.2	IR Storing and Emulating	26
6.3	PN532 Firmware	27
6.3.1	Reading the UID	27
6.3.2	Writing a Message on the NFC/RFID Card	28
6.4	CC1101	29
6.4.1	Receiving RF Signal	29
6.4.2	Register Check	29
7	Issues Faced	31
7.1	Issues Faced in the RFID/NFC	31
7.2	Issues Faced in the IR	31
7.3	Issues Faced in the CC1101	31
8	Conclusion	33

9 Future Prospect	34
--------------------------	-----------

References	35
-------------------	-----------

1. Introduction

1.1 About

Platypulse is a compact multifunctional embedded platform inspired by the concept of the Flipper Zero. It integrates three widely used communication technologies: RF, RFID/NFC, and IR. The system is designed to receive, transmit, store, and analyze supported signals. It combines custom hardware and embedded firmware into a unified platform for wireless experimentation. Platypulse provides practical insight into signal communication, embedded systems, and protocol behavior. The project aims to make invisible wireless interactions understandable through hands-on exploration and innovation.

1.2 Problem Statement

The widespread use of wireless communication technologies such as RF, RFID/NFC, and IR has created a need for practical platforms that allow these technologies to be studied and understood in an integrated manner. Currently, different communication protocols often require separate hardware, interfaces, and software environments, resulting in fragmented development and experimentation. This makes it challenging for engineering students and developers to understand, test, and manage multiple communication technologies within a single system. Additionally, the complexity of integrating different communication modules, signal formats, and data-handling mechanisms presents significant hardware and firmware development challenges. Therefore, there is a need for a unified embedded platform that addresses these challenges while providing a structured environment for learning, experimentation, and development.

1.3 Solution

Platypulse is developed as a compact, integrated embedded platform that brings RF, RFID/NFC, and IR communication capabilities together within a single system. The platform incorporates dedicated communication modules, a microcontroller, data storage, and a unified user interface to manage different communication operations. It is designed to support the reception, processing, storage, analysis, and transmission of supported signals. The system combines custom PCB design with embedded firmware to provide reliable communication between the hardware modules and the central controller. Its modular architecture also provides a foundation for future expansion and integration of additional communication technologies.

2. Custom PCB Layout

2.1 Introduction

This chapter describes the design and layout of the custom PCB developed for the Platypulse project. The primary objective of the PCB design was to integrate the ESP32, PN532, CC1101, IR receiver and transmitter circuitry, and TFT display interface onto a compact and reliable single-board platform. The design focuses on proper signal routing, stable power distribution, efficient component placement, and reliable communication between the different subsystems. All component connections, footprints, and electrical requirements were designed and verified with reference to the respective component datasheets and technical documentation.

2.2 List of Components

The following components are used in the Platypulse PCB:

- **ESP32-S3-WROOM-1**: Main microcontroller module responsible for system control, signal processing, peripheral communication, and overall operation of the Platypulse.
- **PN5321A3HN_C1xx**: NFC/RFID controller IC used for communication with supported NFC and RFID tags and devices.
- **CC1101**: Low-power RF transceiver IC used for receiving and transmitting supported radio-frequency signals.
- **USB Type-C Receptacle USB2.0 16P**: Provides the primary USB interface for power supply and communication with the ESP32-S3.
- **AMS1117-3.3**: Linear voltage regulator used to regulate the 5 V input from USB Type-C to a stable 3.3 V supply required by the system circuitry.
- **TSOP382xx**: Infrared receiver used to detect and demodulate IR signals from compatible transmitters and remote-control devices.
- **IR LED**: Infrared transmitter used to transmit IR signals generated and controlled by the ESP32-S3.

2.3 Circuit Connection

2.3.1 ESP32-S3-WROOM-1 Connections

The ESP32-S3-WROOM-1 acts as the central microcontroller and is connected to every peripheral module through its GPIO matrix. Its GPIOs are routed as follows:

- **SPI bus (shared)** — MOSI, MISO, SCK, and individual CS lines connect to the CC1101, NRF24L01, and Display TFT modules, allowing multiple SPI devices to share the same bus while being selected individually through their chip-select pins.

- **I2C lines** — routed to the Joystick and PN532 (NFC) modules for control and data exchange.
- **GPIO0 and EN (RESET)** — connected to the Boot and Enable switches, allowing manual entry into flashing mode and manual reset during firmware upload.

Decoupling capacitors are placed close to the 3V3 pins of the module to suppress high-frequency switching noise and provide a stable supply during peak current draw.

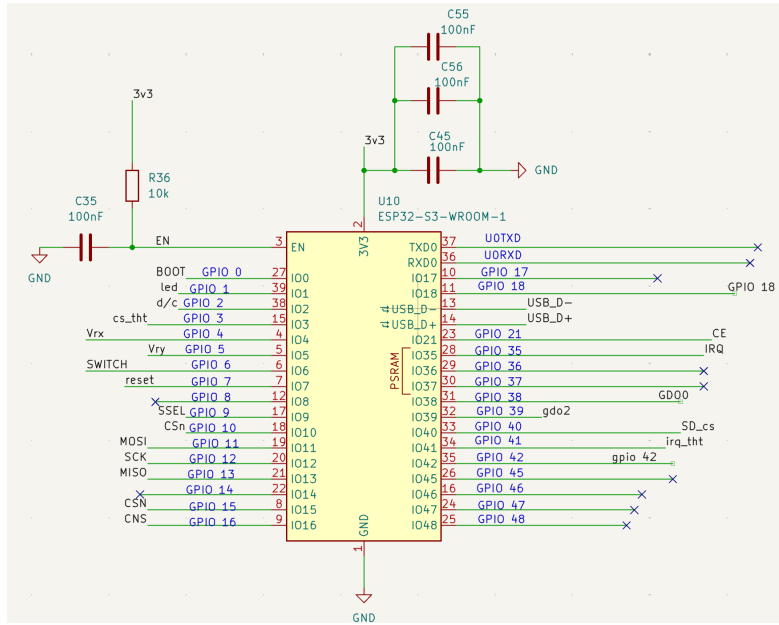


Figure 2.1: ESP32-S3-WROOM-1 schematic and module

Reference: *ESP32-S3-WROOM-1*

https://documentation.espressif.com/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf

2.3.2 CC1101 (Sub-GHz Transceiver) Connections

The CC1101 communicates with the ESP32-S3 over SPI (SCK, MOSI, MISO, CSn). The GDO0 and GDO2 pins are used as interrupt/status lines to notify the microcontroller of packet reception or transmission events, while also matching impedance to minimize reflection and maximize transmitted power.

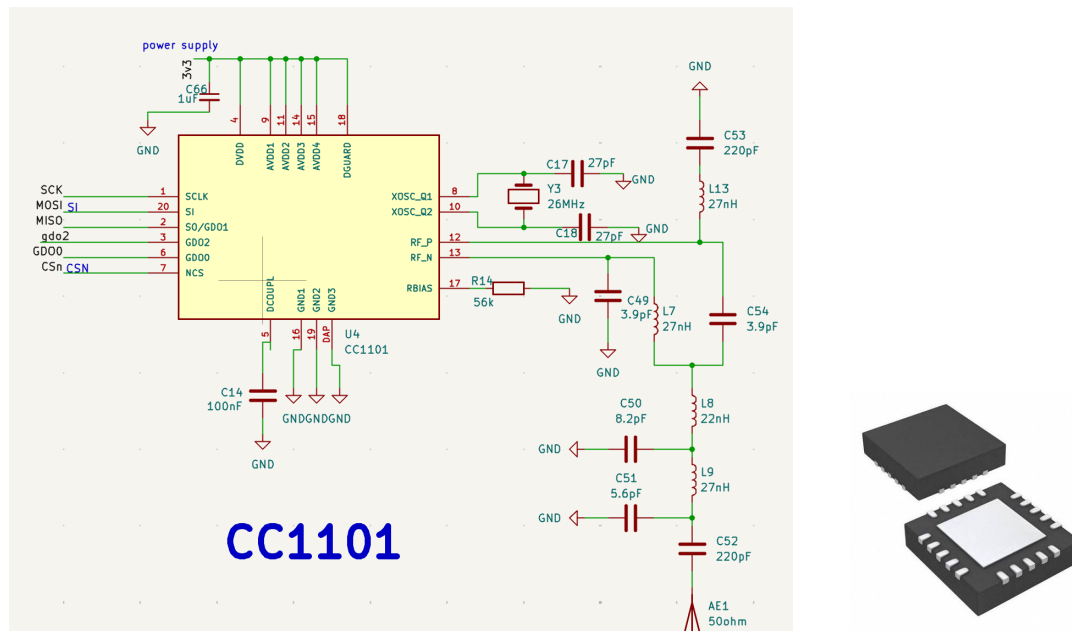


Figure 2.2: CC1101 schematic and chip

<https://www.ti.com/lit/ds/symlink/cc1101.pdf>

2.3.3 IR Transmitter and TSOP38238 (IR Receiver) Connections

The IR Transmitter circuit uses a transistor to drive an IR LED, controlled by a GPIO pin from the ESP32 through a current-limiting resistor, enabling the board to transmit infrared remote-control signals. The TSOP38238 is a dedicated IR receiver module that demodulates incoming 38kHz-modulated IR signals and outputs a clean digital signal directly to a GPIO pin of the ESP32, enabling the board to also receive and decode IR remote signals.

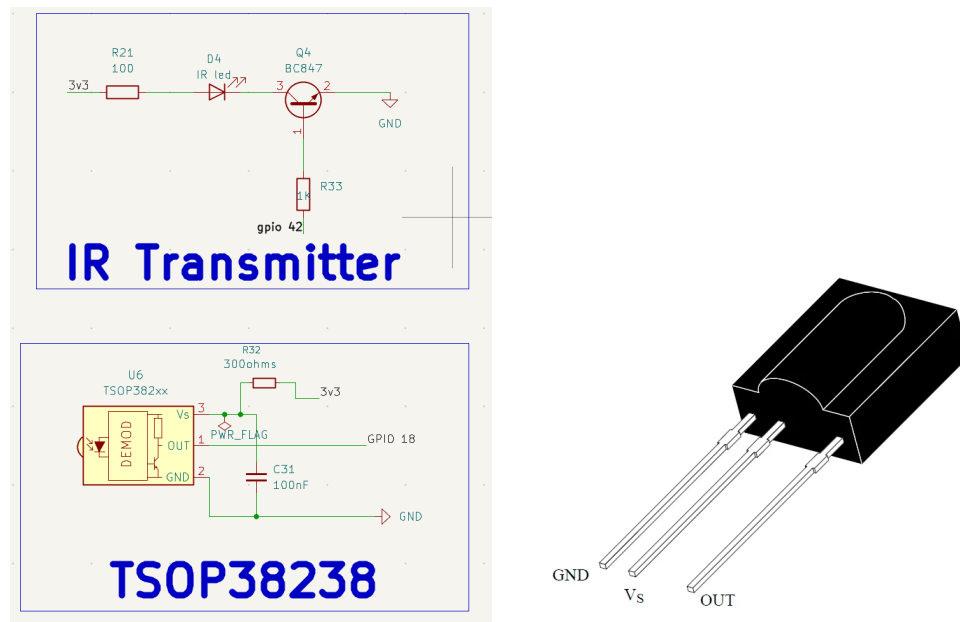


Figure 2.3: IR Transmitter and TSOP38238 schematic, with TSOP38238 pinout

<https://www.vishay.com/docs/82490/tsop321.pdf>

Tsop38238

2.3.4 PN532 Connections

The PN532 module communicates with the ESP32-S3 using the SPI (Serial Peripheral Interface) protocol, with the communication interface selected through the module's SEL pins. A 27.12MHz crystal oscillator provides the accurate clock reference required by the PN532 for reliable NFC operation. The PN532 is connected to an external antenna coil through matching capacitors, forming an LC resonant circuit tuned to the 13.56 MHz NFC operating frequency. This enables the module to generate and detect the RF field required for NFC card and tag communication.

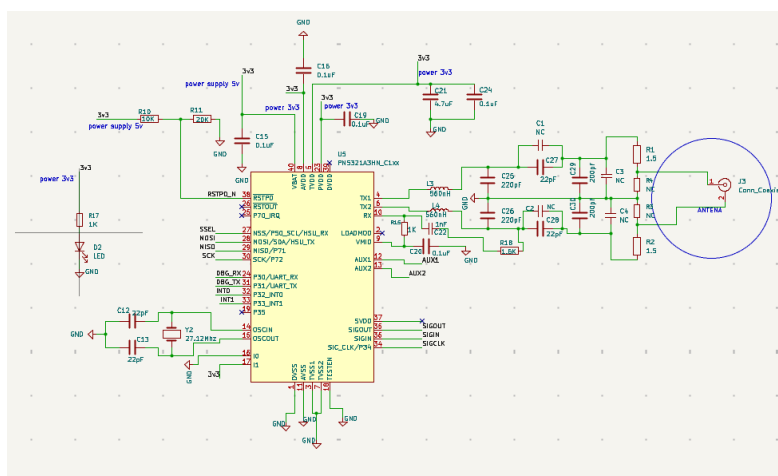


Figure 2.4: PN532 schematic and chip

https://osoyoo.com/driver/PN532_datasheet.pdf

2.3.5 Voltage Regulator Connections

The main power supply enters through the battery/USB connector and is regulated by the AMS1117-3.3 voltage regulator. The VIN pin is connected to the PWR_FLAG net, which supplies an unregulated input voltage in the 3.7 V–5.5 V range from the battery or USB source. This voltage is regulated down to a stable 3.3 V output, which powers the ESP32-S3, CC1101, nRF24L01, PN532, and all other peripheral modules on the board. A 10 μ F capacitor is placed on the input side and a 22 μ F capacitor on the output side to filter transients, reduce voltage fluctuations, and ensure a stable power supply during load switching.

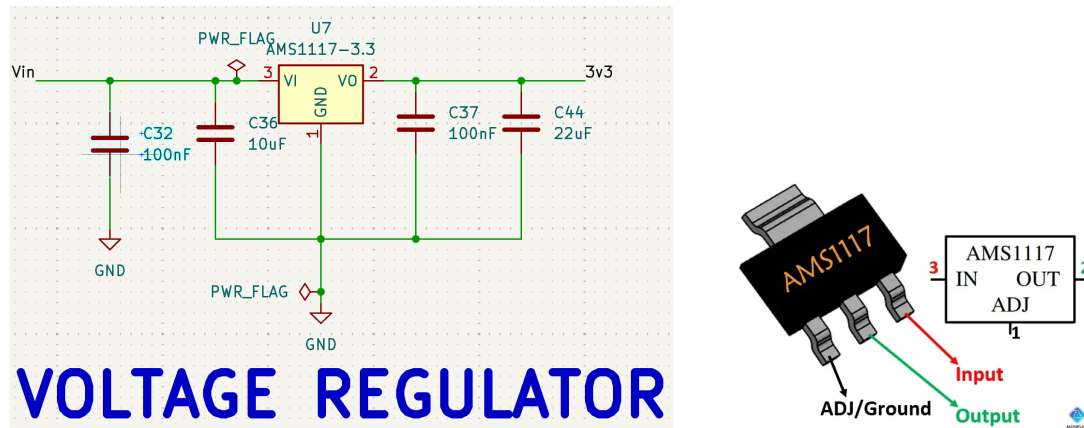


Figure 2.5: Voltage regulator schematic and AMS1117 pinout

<https://www.rcscomponents.kiev.ua/datasheets/AMS1117-datasheet.PDF>

2.3.6 Display TFT Connections

The TFT display is connected to the ESP32-S3 through a dedicated display header, which provides the required interfaces for the TFT display, touch controller, and SD card. The display uses the SPI interface, including SCK, MOSI, MISO, and CS signals, along with dedicated DC (Data/Command) and RST (Reset) control lines. The touch controller and SD card also communicate with the ESP32-S3 through SPI, using their respective chip-select and control signals. This arrangement allows the ESP32-S3 to share the SPI bus between the display, touch interface, and SD card while independently selecting each peripheral through its dedicated CS line.



Figure 2.6: TFT display module

<https://www.hpinfotech.ro/ILI9486.pdf>

2.3.7 Joystick Connections

The joystick module outputs analog voltages proportional to its X and Y axis positions, along with a digital output for the push-button (SEL) function. These are connected to the ADC-capable GPIOs and a digital GPIO of the ESP32 respectively, enabling analog position reading and button press detection.

2.3.8 Modular RF Interface

The PCB is designed with a flexible RF interface to support different RF modules according to the required communication protocol and operating frequency. The RF module communicates with the ESP32 through the SPI interface, while dedicated control and chip-select lines are provided for independent module selection and operation. This modular approach was intentionally adopted in the PCB design to provide flexibility, reusability, and future expandability. Different RF modules can be accommodated on the same hardware platform, allowing the system to support different wireless protocols and frequency bands without requiring a major change to the main PCB architecture.

2.4 Routing

The PCB routing was carried out with special attention to RF performance, signal integrity, and module isolation. Ground stitching vias were provided at suitable locations to improve grounding and reduce unwanted RF coupling. Special care was taken around the SMA and U.FL connectors, keeping components and unnecessary copper away from the antenna regions. Dedicated antenna keep-out areas were maintained to minimize interference and parasitic effects.

The PN532, CC1101, and IR sections were placed in different regions of the PCB to provide better physical and electrical isolation. The PN532 section is positioned toward the center of the PCB for the NFC functionality. The CC1101 RF section is placed on the left side, with appropriate consideration for its RF routing and antenna connection.

The IR circuitry is placed on the right side of the PCB, separated from the other RF sections.

A TFT display header is provided near the center of the PCB for convenient display connectivity, including touch and SD-card interfaces.

The ESP32-S3 is positioned toward the upper region to provide efficient connectivity to the different peripheral sections.

Overall, the routing and component placement were designed to minimize interference, maintain proper RF clearances, and provide reliable operation of all communication modules.

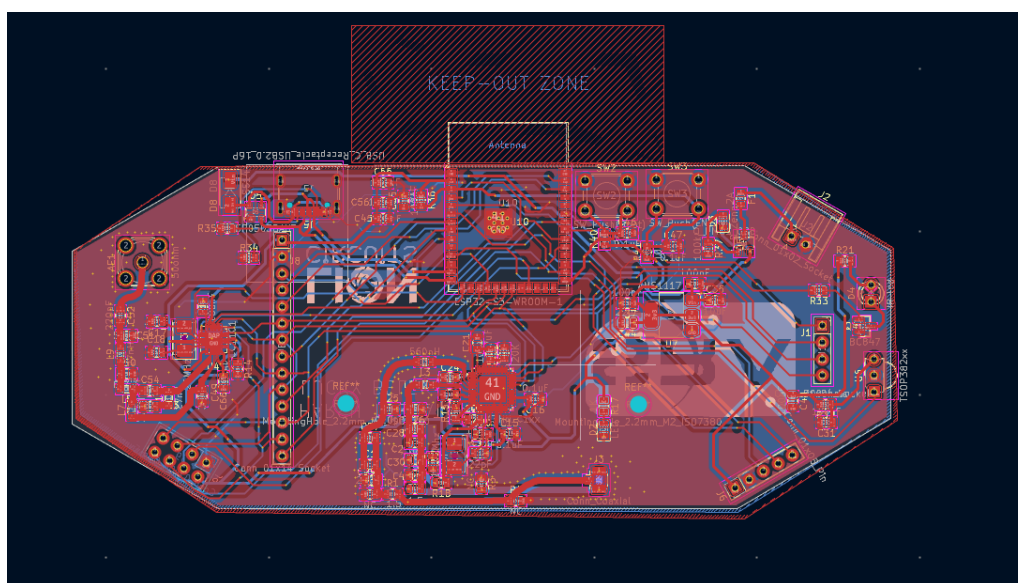


Figure 2.7: PCB routing layout

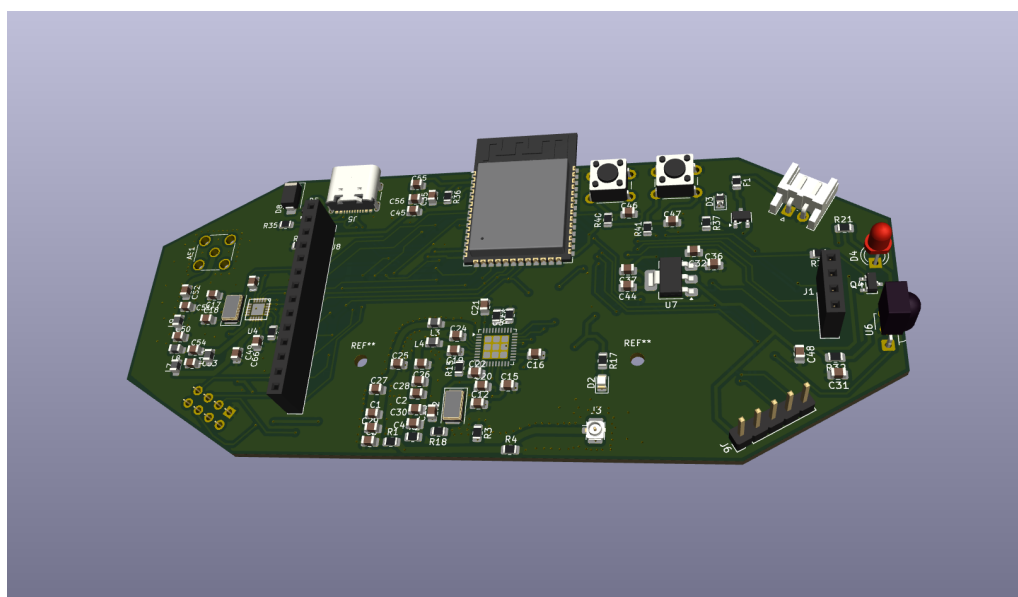


Figure 2.8: 3D viewer of the PCB

3. Wireless Communication

3.1 Introduction

Wireless communication modules refer to hardware interfaces that allow a device to exchange data with the surrounding environment — whether through RF signals, NFC fields, or infrared pulses — without requiring a direct wired connection to the target device. This approach directly addresses our problem statement of repetitive hardware connections, extending it beyond firmware updates to the way the board interacts with external systems. By leveraging modules such as the CC1101, PN532, and IR transceiver, we can seamlessly capture, interpret, and retransmit signals from a wide range of existing wireless devices, enabling faster, more flexible, and largely contactless interaction with minimal manual effort.

3.2 CC1101 — Sub-1 GHz RF Transceiver

3.2.1 Communication Workflow

The CC1101 connects to the microcontroller over SPI. Before any data can be sent or received, the relevant configuration registers — operating frequency, modulation type, data rate, and packet parameters — must be written to the chip.

Transmit path: packet formation → data encoding (NRZ or Manchester) → CRC generation → modulation (FSK, GFSK, or OOK) → RF transmission via antenna.

Receive path: demodulation → decoding → CRC verification → packet validity check → data delivered to the microcontroller.

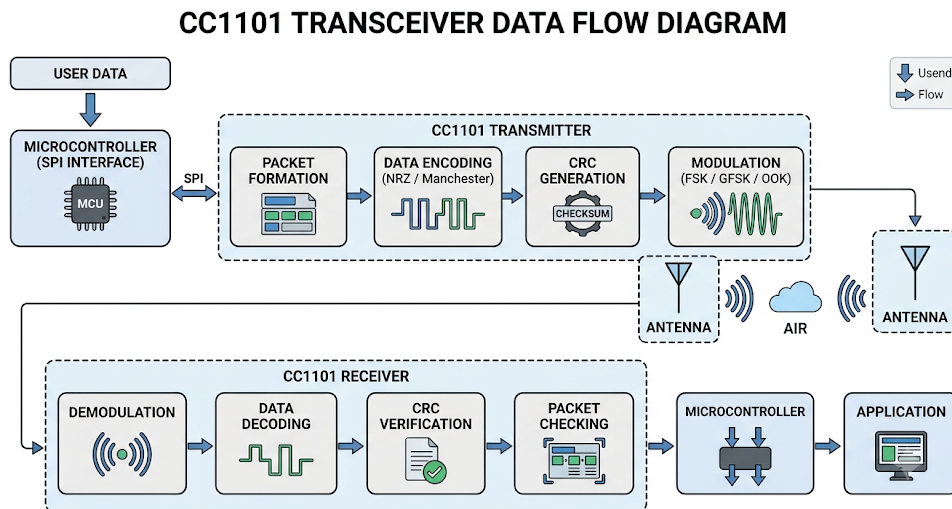


Figure 3.1: CC1101 transceiver data flow diagram

3.2.2 Register Access over SPI

Every SPI transaction begins with a single header byte that encodes three things:

- **R/W bit** — whether the transaction is a read or a write
- **Burst bit (B)** — whether one register or several consecutive registers are being accessed
- **6-bit address** — the target register

In burst mode, the CC1101 automatically increments the register address after each byte, so multiple consecutive registers can be read or written in one transaction instead of many.

3.2.3 Modulation and Line Encoding

The CC1101 uses FSK (Frequency Shift Keying), where data is represented by shifting the carrier between two discrete frequencies rather than switching it on/off (ASK) or shifting its phase (PSK). FSK is comparatively resilient to noise, walls, and distance, and unlike PSK does not require precise phase synchronization between transmitter and receiver, making it a practical default for low-power sub-GHz links.

Before modulation, the bitstream is line-encoded using NRZ-L (Non-Return-to-Zero, Level):

- A high level represents a 1
- A low level represents a 0
- The line holds its level until the next bit forces a change

A well-defined encoding/decoding scheme is what makes the link robust: both ends follow the same fixed rule, so the receiver can correctly distinguish 0s and 1s even under noise or attenuation, and can recover accurate bit timing rather than guessing where one bit ends and the next begins.

3.2.4 Error Checking

The CC1101 generates a Cyclic Redundancy Check (CRC) on transmission and verifies it on reception entirely in hardware. If the check fails, the packet is treated as corrupted and discarded.

3.3 PN532 — NFC/RFID Module

3.3.1 Frame Structure

Every command or response exchanged with the PN532 follows a fixed frame format:

Byte	Meaning
00	Preamble — marks the start of the frame
00 FF	Start code — marks the start of a valid packet
LEN	Number of data bytes that follow (TFI + PD0...PDn)
LCS	Length checksum
TFI	Frame identifier — direction of transfer (D4 = host → PN532, D5 = PN532 → host)
PD0...PDn	Command code followed by its parameters
DCS	Data checksum
00	Postamble

3.3.2 Checksums

Aspect	LCS (Length Checksum)	DCS (Data Checksum)
Verifies	The LEN byte	The data bytes (TFI + command + parameters)
Formula	$LEN + LCS \equiv 0 \pmod{256}$	$TFI + PD0...PDn + DCS \equiv 0 \pmod{256}$
On failure	The frame's length can't be trusted — packet discarded	The payload is corrupted — packet rejected

3.3.3 NFC Data Handling and RFID Access-Control Observations

NFC/RFID technology supports both data storage and device interaction through compatible tags and readers. The PN532 can write supported data to writable NFC/RFID tags, allowing information such as URLs, contact details, text, and other NDEF records to be stored on the tag. When such a tag is brought near an NFC-enabled smartphone, the phone can detect and interpret the stored information, for example, opening a website or displaying contact information. NFC can also be used for other applications such as triggering predefined actions, sharing information, or storing configuration data, depending on the tag, smartphone, and application being used.

This functionality is different from RFID-based access-control systems. Some door-access systems identify a card primarily through its UID or another card identifier and compare the presented identifier with the identifiers authorized by the access-control system. Some commercially available cards can present a configurable UID; however, whether such a card works with a particular access-control system depends on the reader's authentication method and whether it checks only the identifier or also verifies additional card data and cryptographic authentication. This should also be distinguished from contactless payment cards. Modern payment cards use an EMV chip along with dedicated payment-security mechanisms, so simply copying or changing an NFC/RFID identifier does not reproduce the functionality of a genuine payment card.

Overall, NFC can be used for storing readable information and supporting device interactions, while RFID access-control systems may use identifiers together with additional authentication mechanisms for authorization.

3.4 IR Signal Learning, Storage, and Emulation

3.4.1 Signal Learning / Receiving

The TSOP infrared receiver listens to an existing remote control and captures the IR signal whenever a button is pressed. Since different devices and manufacturers can use different IR protocols and command formats, the system learns the actual signal produced by the specific remote rather than assuming a single universal protocol. For example, two different air-conditioner models may use completely different signal structures for similar functions such as temperature control.

3.4.2 Storing and Emulating

Once the signal is received, the system stores the captured IR data so that the learned command can be recalled later. Multiple commands can be stored for different devices and functions, allowing the system to build a collection of learned remote-control signals.

During emulation, the stored signal is reproduced through the IR LED, which transmits the corresponding infrared pattern toward the target device. In this way, the system can learn commands from an original remote, store them for later use, and emulate the remote when required, effectively providing a learning-based universal IR interface.

4. Wired Communication

4.1 SPI (Serial Peripheral Interface)

SPI (Serial Peripheral Interface) is the high-speed communication backbone used in Platypulse to connect the ESP32-S3 with several important peripherals. It is a synchronous, full-duplex protocol, meaning the ESP32-S3 provides the clock and can transmit and receive data at the same time.

SPI uses four primary signals:

- **SCK (Serial Clock)**: Generated by the ESP32-S3 to synchronize data transfer.
- **MOSI (Master Out Slave In)**: Carries data from the ESP32-S3 to the selected peripheral.
- **MISO (Master In Slave Out)**: Carries data from the peripheral back to the ESP32-S3.
- **CS/SS (Chip Select)**: Selects which peripheral should communicate with the ESP32-S3. The selected device is activated by pulling its CS line LOW.

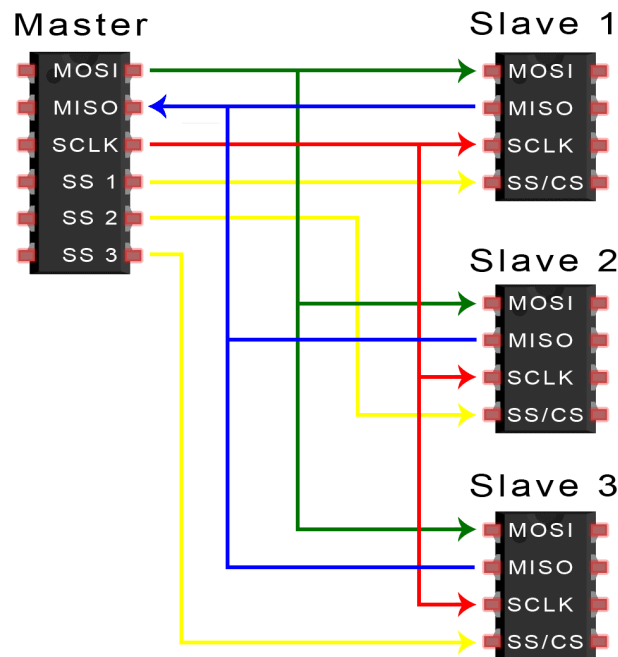


Figure 4.1: SPI Master with multiple Slave devices

In the Platypulse hardware, the ESP32-S3 acts as the SPI master, while the TFT display, PN532 NFC module, and CC1101 RF transceiver operate as SPI peripherals. The common SPI lines, such as SCK, MOSI, and MISO, can be shared among these peripherals, while each device has its own dedicated CS (Chip Select) line.

This arrangement allows the ESP32-S3 to use a single SPI interface for multiple modules while communicating with each device independently. Whenever the ESP32-S3 needs to communicate with a particular peripheral, it selects the corresponding device by pulling its CS line LOW and then performs the required data transfer.

Thus, SPI provides Platypulse with a fast, efficient, and organized communication interface between the main controller and its display, NFC, and RF subsystems.

4.2 I2C (Inter Integrated Circuit)

I2C is the synchronous communication means at a time it can communicate multi slave as well it can have multi master communication. I2C uses only two wires for communication, no matter how many devices are attached to the bus. SDA: The line for master and slave to send data. SCL: The line carrying the clock pulses generated by the Master.

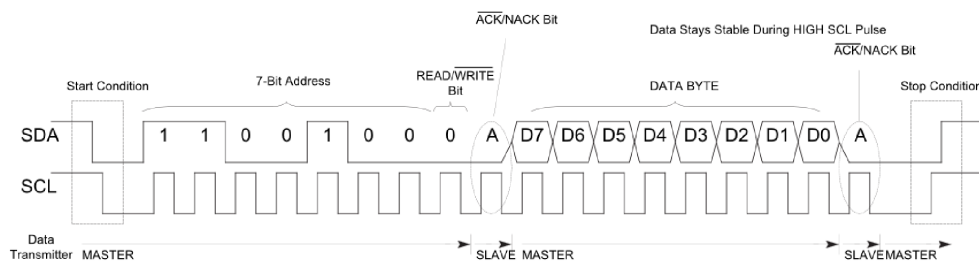


Figure 4.2: I2C communication timing diagram

Multiple devices share the same lines, I2C uses addresses to target specific devices. Each chip has a unique built-in address. The Master sends a start condition, followed by the 7-bit address of the slave it wants to talk to, plus a Read/Write bit. The matching slave responds with an ACK (Acknowledge) bit. Data packets (8 bits each) are then transmitted, followed by a Stop condition. Supports up to 128 devices on the same two wires. We have started implementing the I2C configuration to the JOYSTICK.

5. Software Defined Radio

5.1 Introduction

Software Defined Radio (SDR) provides the link between the physical RF world and digital computation. In wireless systems, signals exist naturally as continuous analog waveforms, but digital systems need these signals to be represented as numerical data for processing and analysis. In Platypulse, SDR concepts were used primarily for debugging and analyzing the CC1101 RF communication, allowing us to observe the actual RF behavior of the system rather than relying only on software-level information.

5.2 Receiving The Signal

5.2.1 Introduction

The GNU Radio flowgraph is designed to generate a continuous On-Off Keying (OOK) signal from a predefined byte pattern and transmit it at 433.92 MHz using an SDR. The data passes through several stages, where the original bytes are converted into individual bits, stretched into time-domain pulses, converted into complex I/Q samples, filtered, and finally transmitted as an RF signal.

5.2.2 Vector Source

Continuously generates the required byte pattern, such as 0xAA, 0xAB, 0xAB.

5.2.3 Repack Bits

Converts each 8-bit byte into individual 0 and 1 bits for OOK modulation.

5.2.4 Repeat

Repeats each bit 120 times. At a 2.4 MHz sample rate, this produces a 50 μ s bit duration, corresponding to a 20 kbps data rate.

5.2.5 Char to Float

Converts the binary values from 0/1 into floating-point values 0.0/1.0 for further signal processing.

5.2.6 Constant Source

Provides a constant 0 for the imaginary (Q) component.

5.2.7 Float to Complex

Combines the bitstream and zero-valued Q component into complex I/Q baseband samples.

5.2.8 Multiply Const

Scales the signal amplitude by 0.6 to maintain a controlled signal level.

5.2.9 Low Pass Filter

Smooths the OOK transitions and reduces unwanted high-frequency spectral components.

5.2.10 Osmocom Sink

Sends the processed I/Q samples to the SDR, which transmits the signal at 433.92 MHz.

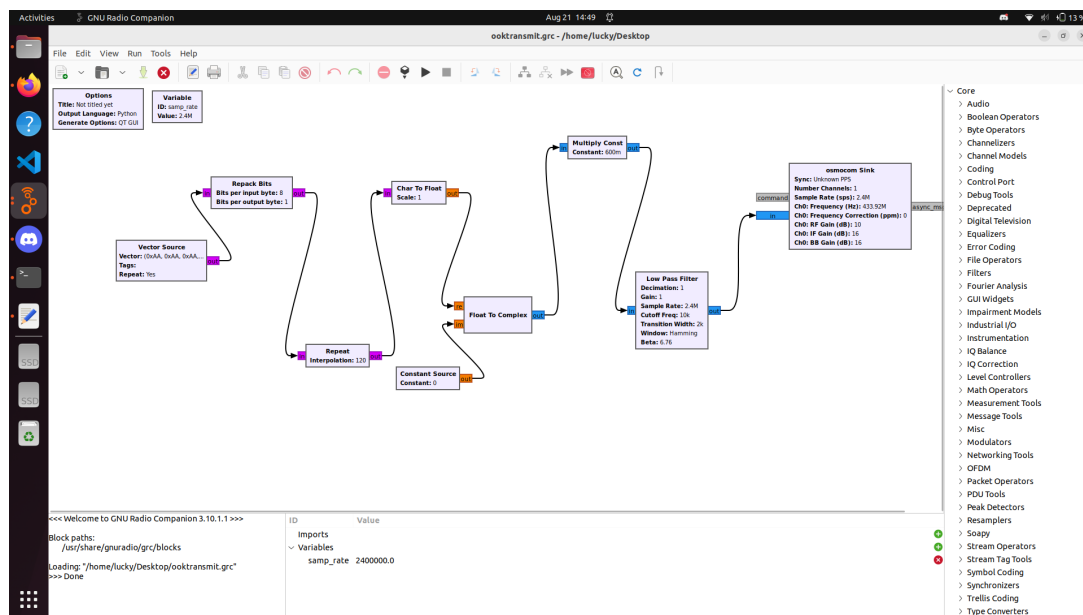


Figure 5.1: GNU Radio Companion flowgraph for generating and transmitting the OOK signal

5.3 Transmitting the Signal

5.3.1 Introduction

The OOK receiver flowgraph is designed to capture and recover the digital data transmitted at 433.92 MHz. The RTL-SDR/HackRF first captures the RF signal as complex IQ samples, which are then filtered and converted into an amplitude envelope. The recovered signal is normalized, thresholded into binary bits, and finally reconstructed into bytes for synchronization and further analysis.

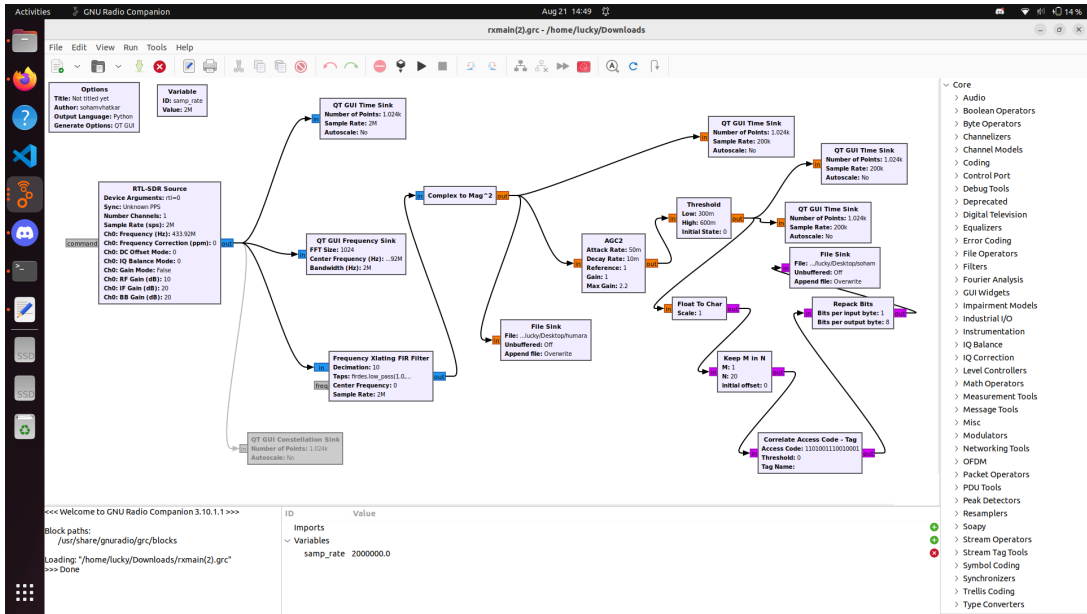


Figure 5.2: GNU Radio Companion flowgraph for receiving and decoding the OOK signal

5.3.2 RTL-SDR Source

Captures complex IQ samples around 433.92 MHz at a 2 MHz sample rate with manual gain control.

5.3.3 QT GUI Frequency Sink

Displays the received frequency spectrum, allowing the presence and bandwidth of the OOK signal to be observed.

5.3.4 QT GUI Time Sink

Shows the raw received IQ waveform in the time domain.

5.3.5 QT GUI Constellation Sink

Displays the I/Q samples on a complex plane. For OOK, the samples mainly represent the carrier OFF and ON amplitude states.

5.3.6 Frequency Xlating FIR Filter

Isolates the required channel, applies low-pass filtering, and decimates the sample rate from 2 MHz to 200 kHz.

5.3.7 File Sink

Stores the filtered IQ samples for later analysis and debugging.

5.3.8 Complex to Mag²

Performs envelope detection by calculating $I^2 + Q^2$, converting the received OOK signal into an amplitude-based waveform.

5.3.9 QT GUI Time Sink (Envelope)

Displays the recovered envelope, where the ON and OFF states of the transmitted bits can be observed.

5.3.10 AGC2

Automatically adjusts the envelope amplitude to compensate for changes in received signal strength.

5.3.11 File Sink (Normalized)

Saves the normalized envelope signal for further analysis.

5.3.12 Threshold

Converts the analog envelope into clean binary 0 and 1 values using upper and lower thresholds with hysteresis to reduce noise-induced transitions.

5.3.13 QT GUI Time Sink (Digital)

Displays the resulting digital bitstream after threshold detection.

5.3.14 Float to Char

Converts the floating-point binary samples into character/byte-type samples for bit processing.

5.3.15 Keep M in N

Reduces the oversampled bitstream by keeping one representative sample from each bit period, recovering the original bit timing.

5.3.16 Repack Bits (Receiver)

Combines the individual bits back into 8-bit bytes, reversing the bit unpacking performed at the transmitter.

5.3.17 QT GUI Time Sink (Byte Stream)

Monitors the recovered byte stream.

5.3.18 Correlate Access Code - Tag

Searches for the defined synchronization pattern 1101001110010001 and identifies the beginning of a valid data frame.

6. Firmware

6.1 Introduction

PLATYPULSE firmware is the core software that controls and manages the functionality of the PLATYPULSE device. It is developed using the ESP-IDF framework for the ESP32-S3 microcontroller. The firmware provides the software interface between the hardware components and the user, enabling the device to perform its intended operations. It is designed to provide reliable, modular, and expandable control of the overall PLATYPULSE system.

6.2 IR Firmware

6.2.1 IR Receiving

```
void app_main(void)
{

    gpio_config_t io_conf = {
        .pin_bit_mask = (1ULL << IR_PIN),
        .mode = GPIO_MODE_INPUT,
        .pull_up_en = GPIO_PULLUP_ENABLE, // Ensures pin stays HIGH when idle
        .pull_down_en = GPIO_PULLDOWN_DISABLE,
        .intr_type = GPIO_INTR_DISABLE
    };
    gpio_config(&io_conf);

    while (1) {
        // TSOP outputs LOW (0) when it detects a 38kHz IR light burst
        if (gpio_get_level(IR_PIN) == 0) {
            printf("ir signal mil gaya \n");

            // Short delay to avoid spamming the console while the button is held
            vTaskDelay(pdMS_TO_TICKS(200));
        } // Poll every 10ms to stay responsive
        vTaskDelay(pdMS_TO_TICKS(10));
    }
}
```

The code first configures the IR receiver GPIO as an input with an internal pull-up resistor. It then continuously monitors the pin, where a LOW signal indicates that the TSOP has detected an IR signal. When detected, it prints a message.

6.2.2 IR Storing and Emulating

```
while (1) {
    if (stored_symbol_count == 0) {

        rmt_receive_config_t receive_config = {
            .signal_range_min_ns = 1250,
            .signal_range_max_ns = 40000000,
        };
        ESP_ERROR_CHECK(rmt_receive(rx_chan, temp_symbols,
            sizeof(temp_symbols), &receive_config));
        ESP_LOGW(TAG, "RECORD MODE: Point remote at TSOP and press a
button...");
        rmt_rx_done_event_data_t rx_data;
        if (xQueueReceive(rx_queue, &rx_data, portMAX_DELAY)) {

            stored_symbol_count = rx_data.num_symbols;
            memcpy(stored_symbols, temp_symbols, stored_symbol_count *
sizeof(rmt_symbol_word_t));

            ESP_LOGI(TAG, "Signal saved! Captured %d symbols.",
stored_symbol_count);

            rmt_disable(rx_chan);
        }
    }
    else {

        ESP_LOGI(TAG, "EMITTING stored signal...");
        rmt_transmit_config_t tx_config = {
            .loop_count = 0,
        };

        ESP_ERROR_CHECK(rmt_transmit(tx_chan, copy_encoder, stored_symbols,
stored_symbol_count * sizeof(rmt_symbol_word_t), &tx_config));
        ESP_ERROR_CHECK(rmt_tx_wait_all_done(tx_chan, -1));

        // Wait 3 seconds before emitting again
        vTaskDelay(pdMS_TO_TICKS(3000));
    }
}
```

The firmware initializes separate RMT RX and TX channels, where the RX channel captures the IR pulse timings from the TSOP and stores them as `rmt_symbol_word_t` data.

After capture, the stored RMT symbols are passed to the RMT TX channel through a copy encoder, with a 38 kHz carrier applied to reproduce the original IR waveform through the IR LED.

6.3 PN532 Firmware

6.3.1 Reading the UID

```
while (1) {
    // The pn532_wait_for_card() function now encapsulates IRQ arming and
    waiting.

    if (pn532_wait_for_card() == ESP_OK) {
        ESP_LOGI(TAG, GREEN "Interrupt received!" RESET);
        uint8_t uid[7];
        uint8_t uidLength;
        if (pn532_read_detected_passive_target(uid, &uidLength) == ESP_OK) {

            char uid_str[uidLength * 2 + 1];
            for (int i = 0; i < uidLength; i++) {
                sprintf(&uid_str[i * 2], "%02X", uid[i]);
            }
            uid_str[uidLength * 2] = '\0';
            ESP_LOGI(TAG, "Card detected. UID: %s", uid_str);
            if (strcmp(uid_str, "C5AC0907") == 0) {
                led_state = !led_state;
                gpio_set_level(GPIO_NUM_2, led_state);
            }

        } else {
            ESP_LOGW(TAG, "Interrupt received, but the subsequent card reading
            failed.");
        }

        vTaskDelay(pdMS_TO_TICKS(500));
    } else {

        ESP_LOGE(TAG, RED "Failed to arm the reader for detection. Trying to
        recover..." RESET);
        vTaskDelay(pdMS_TO_TICKS(RECOVERY_DELAY_MS));

        if (pn532_init() != ESP_OK){
            ESP_LOGI(TAG, RED "pn532_init failed" RESET);
        } else {
            ESP_LOGI(TAG, GREEN "pn532_init succeeded" RESET);
        }
        if (pn532_soft_reset() != ESP_OK){
            ESP_LOGI(TAG, RED "pn532_soft_reset failed" RESET);
        } else {
            ESP_LOGI(TAG, GREEN "pn532_soft_reset succeeded" RESET);
        }
    }
}
```

```
if (pn532_sam_config() != ESP_OK){
  ESP_LOGI(TAG, RED "pn532_sam_config failed" RESET);
} else {
  ESP_LOGI(TAG, GREEN "pn532_sam_config succeeded" RESET);
}
trigger_isr();
}
```

When a card is detected, the PN532 generates an interrupt, after which the program reads the card's UID as raw byte data. The UID is then converted into a hexadecimal string for easy readability. Since each byte is represented by two hexadecimal characters, the required string size is calculated as $\text{uidLength} \times 2 + 1$, where the extra +1 is reserved for the null terminator (`\0`) that marks the end of the C string. Each element of the UID array, such as `uid[0]`, `uid[1]`, and so on, represents one byte and is converted into its corresponding two-character hexadecimal value.

If the card reading fails after the interrupt is received, the program displays a warning message indicating that the interrupt was received but the card could not be read successfully. If the PN532 fails to arm itself for card detection, the program enters a recovery process. It attempts to reinitialize the PN532, perform a soft reset, reconfigure the SAM (Secure Access Module) configuration, and trigger the interrupt again. The program also logs the success or failure of each recovery step, making it easier to identify where the initialization or recovery process failed.

6.3.2 Writing a Message on the NFC/RFID Card

```
void loop() {
  uint8_t uid[7];
  uint8_t uidLength;
  if (nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A, uid, &uidLength)) {
    Serial.println("Tag detected! Writing URL...");
    if (writeNDEFURL("sravjti.in/")) {
      Serial.println("URL written successfully!");
    } else {
      Serial.println("Failed to write URL.");
    }
    delay(3000);
  }
  delay(200);
}
```

This code uses the PN532 NFC reader to detect an NTAG21x tag and write a URL into its memory using the NDEF format. After detecting the tag, the program takes the URL as a string and stores its characters as byte values, which are commonly represented in hexadecimal. It then creates the required NDEF URI record, adds the TLV structure required by the tag, and divides the resulting data into 4-byte blocks called pages, since each NTAG page contains 4 bytes. The program starts writing from page 4 to avoid the reserved memory area containing tag information such as the UID and configuration

data. Each 4-byte page is written sequentially using the PN532, and the program reports whether the URL was successfully written or if any page-write operation failed.

6.4 CC1101

6.4.1 Receiving RF Signal

```
static void print_packet(const uint8_t *buf, size_t len)
{
    printf("\n===== DATA PACKET RECEIVED =====\n");
    printf("Length: %d bytes\n", (int)len);
    printf("HEX : ");
    for (size_t i = 0; i < len; i++) printf("%02X ", buf[i]);
    printf("\n");
    printf("ASCII: ");
    for (size_t i = 0; i < len; i++) {
        char c = (char)buf[i];
        printf("%c", (c >= 32 && c <= 126) ? c : '.');
    }
    printf("\n===== \n\n");
}
```

This code configures the CC1101 RF transceiver as an ASK/OOK packet receiver using the ESP32-S3. It initializes the SPI interface and resets the CC1101 before configuring it to operate at 433.92 MHz. The CC1101 is configured with a specific sync word, fixed 4-byte packet length, and OOK modulation, while CRC and data whitening are disabled. When a complete packet is received, the GDO0 pin generates a falling-edge interrupt, which alerts the ESP32 to read the received data from the CC1101 RX FIFO through SPI. After processing the packet, the RX FIFO is flushed and the CC1101 is placed back into receive mode so that it can continuously listen for the next packet.

6.4.2 Register Check

```
extern "C" void app_main(void)
{
    vTaskDelay(pdMS_TO_TICKS(1000));
    ESP_LOGI("MAIN", "Hello World...?");
    spi_bus_config_t busConfig = {};
    busConfig.mosi_io_num = GPIO_NUM_23;
    busConfig.miso_io_num = GPIO_NUM_19;
    busConfig.sclk_io_num = GPIO_NUM_18;
    busConfig.quadwp_io_num = -1;
    busConfig.quadhd_io_num = -1;
    ESP_ERROR_CHECK(spi_bus_initialize(SPI3_HOST, &busConfig,
    SPI_DMA_DISABLED));
    spi_device_interface_config_t deviceConfig = {};
    spi_device_handle_t cc1101;
    deviceConfig.command_bits = 0;
```

```
deviceConfig.address_bits = 0;
deviceConfig.dummy_bits = 0;
deviceConfig.mode = 0;
deviceConfig.clock_speed_hz = 1000000; // 1 MHz
deviceConfig.spics_io_num = GPIO_NUM_5; // Chip Select Pin
deviceConfig.queue_size = 1; // Program doesn't queue anything, only
synchronous methods used
ESP_ERROR_CHECK(spi_bus_add_device(SPI3_HOST, &deviceConfig,
&cc1101));

// We must reset the system when the power supply is turned on (see datasheet:
19.1 Power-On Start-Up Sequence)
strobe_reset(cc1101);

// Retrieve the PARTNUM register value
transmit_data(
cc1101,
(uint8_t[]){CC1101_STATUS_PARTNUM, CC1101_DUMMY_BYTE},
2, // How many bytes we are sending
"PARTNUM"
);

// Retrieve the VERSION register value
transmit_data(
cc1101, (uint8_t[]){CC1101_STATUS_VERSION, CC1101_DUMMY_BYTE},
2,
"VERSION"
);
}
```

It sends command or register-address bytes to the CC1101 and simultaneously receives the response from the device. The code then reads the PARTNUM and VERSION status registers to verify that the CC1101 is responding correctly and prints the returned values to the ESP32 console. Overall, this is a basic CC1101 communication and hardware verification program rather than a complete RF receiver or transmitter.

7. Issues Faced

7.1 Issues Faced in the RFID/NFC

During the initial NFC testing, we faced an issue with UID reading because the PN532 interrupt (IRQ) pin was not connected. Due to this, the ESP32-S3 could not properly receive interrupt signals from the PN532, resulting in unreliable card detection and UID reading. After connecting the IRQ pin, the issue was resolved.

We then tried writing a message to an NFC sticker but found that it was NDEF-writable, not UID-writable. Since the Atomberg door system identifies cards using their UID, we specifically needed a changeable/writable UID card. The cards we ordered from Robu were listed as writable UID cards, but testing showed that they only supported writing messages/data. We later found that suitable changeable-UID cards were difficult to source in India and would need to be imported.

7.2 Issues Faced in the IR

During IR testing, one of the main challenges was determining the complete data length of the AC remote signal. Initially, the allocated storage was insufficient to capture the entire signal, which caused incomplete data during emulation. We therefore increased the signal storage buffer/memory so that the complete IR sequence could be captured and reproduced correctly.

We also explored a Universal AC Remote GitHub repository to support different AC brands. However, its implementation was not directly suitable for our requirements because it did not include the Blue Star AC protocol that we needed to test. Although the SRA implementation included Blue Star support, it was not straightforward to integrate into our existing system. We also tested the approach with a Carrier AC, but the available protocol implementation was outdated, making it unreliable for current AC models. This highlighted the challenge of maintaining a universal IR implementation when AC protocols and their implementations vary across manufacturers and models.

7.3 Issues Faced in the CC1101

During CC1101 testing, we faced several hardware and testing challenges. The modules we used were non-standard versions with pin headers, which required precise soldering and caused unreliable connections. One was found to be faulty through register-level verification, as it returned invalid/null register values.

Loose connections were another major issue. The extender headers and JST connections required careful alignment and firm positioning to maintain a stable connection. JST connections were also only partially soldered to the jumper wires, making the breadboard-based test setup.

Testing the CC1101 also required a more complex setup. For reception, we used a GNU Radio flowgraph to generate and analyze test signals, while for transmission, we used an RTL-SDR to capture the signal. The HackRF did not provide consistent reception during our testing. We also observed that signal quality varied depending on the physical location in our SRA lab.

Another issue was the absence of a clear power peak in the GNU Radio signal analysis. The CC1101 was powered directly from the Mac, and the USB power source was not ideal for maintaining a stable current supply during transmission. Finally, loose hardware connections occasionally caused slight mismatches in the received and decoded signal, making stable connections and proper power delivery critical for reliable CC1101 testing.

8. Conclusion

Platypulse combines RF, RFID/NFC, and IR communication into one embedded platform, using a custom-designed PCB along with firmware written for the ESP32-S3, CC1101, PN532, and IR modules. The system is able to receive, store, analyze, and transmit signals across all three technologies, with the different modules connected through a shared SPI/I2C bus but kept physically separated on the board to avoid interference.

We designed the hardware and firmware to be modular, so each communication module works on its own without depending too much on the others. This also means the platform can be expanded later to support more protocols or frequency bands without redesigning the whole board.

Apart from the technical side, this project taught us a lot as a team. Designing and routing our own PCB helped us understand how things we studied in theory actually behave on a real board — things like component placement, keeping RF sections isolated, and maintaining clean signal paths are very different when you're not just simulating a circuit. We also ran into a lot of hardware issues, like an unconnected interrupt pin or a CC1101 module that just wouldn't respond, and fixing these required a lot of patience and careful, step-by-step debugging instead of guessing.

Working with our teammates and mentors made a big difference throughout the project. Discussing problems together and troubleshooting as a group helped us make better decisions and get past setbacks faster than we would have on our own. Overall, this project gave us hands-on experience with wireless communication systems and PCB design, and also taught us how important teamwork and patience are when building something like this from scratch.

9. Future Prospect

Looking forward, the current build of Platypulse serves as a strong foundation for further development. Future work will focus on completing the remaining hardware integration, refining the firmware, and expanding the platform's capabilities across all three communication domains.

One of the key directions we want to explore is implementing basic cryptographic capabilities on the PCB, allowing captured and stored data to be encrypted before being saved or transmitted. This would make the platform more suitable for secure communication experiments and add an extra layer of protection to sensitive signal data handled by the system.

On the RFID/NFC side, we aim to work towards UID cloning with a self-writable/changeable UID card, so that captured card identifiers can be reproduced more reliably for testing access-control systems. Alongside this, we also want to explore automated detection and rejection logic for smart door systems — for instance, identifying when the same UID or signal is being transmitted continuously or repeatedly in a short span of time, which could indicate a replay attempt, and using this pattern to automatically flag or reject such attempts rather than allowing continuous unlocking.

For the IR module, instead of only focusing on capturing and storing signals, we plan to improve the firmware so that a user can browse through previously learned signals and manually select a specific command to emulate, rather than the system handling emulation in a fixed or automatic sequence. This would make the IR functionality feel more like a proper universal remote, giving the user direct control over which signal is played back and when.

Beyond these specific goals, we also hope to improve the overall reliability of the CC1101 RF module through better hardware connections and more consistent power delivery, and to continue refining the PCB layout based on the issues faced during this phase. As we gain more experience with RF security concepts, signal analysis, and embedded firmware design, we intend to build on this foundation to make Platypulse a more complete and secure multi-protocol communication platform.

References

1. Documentation used to understand the ICs and modules

- https://documentation.espressif.com/esp32-s3-wroom-1_wroom-1u_datasheet_en.pdf
- <https://www.ti.com/lit/ds/symlink/cc1101.pdf>
- https://osoyoo.com/driver/PN532_datasheet.pdf
- <https://www.buydisplay.com/download/ic/ILI9341.pdf>

2. Resources used to understand the software and firmware

- <https://www.youtube.com/playlist?list=PLu0BPYzTjiHru1KmPThmbY-8rRm3EWvUQ>
- <https://docs.espressif.com/projects/esp-idf/en/v5.2/esp32s3/get-started/>
- <https://wiki.gnuradio.org/index.php/Tutorials>

3. GitHub repositories used

- <https://github.com/nopnop2002/esp-idf-cc1101/tree/main/components/cc1101>
- https://github.com/williamwolf11/ESP32_IRUniversalRemote
- <https://github.com/crankyoldgit/IRremoteESP8266>

4. Inspiration

- <https://flipper.net/>